

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Системы автоматизированного проектирования»

ОТЧЕТ
по лабораторной работе №3
по курсу «Логика»
на тему «Динамические списки.»

Выполнили:

студенты группы 25ВВВ4

Волковец А.С

Пантюхин З.С

Приняли:

к.т.н. Юрова О.В.

к.т.н. Деев М.В.

Пенза 2025

1. Цель работы

Научиться работать с разными структурами данных

2. Лабораторное задание

Задание

1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).

2. * На основе приведенного кода реализуйте структуру данных *Очередь*.

3. * На основе приведенного кода реализуйте структуру данных *Стек*.

3. Ход работы

1. Реализовали приоритетную очередь:

```
--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить элемент с приоритетом
2. Извлечь элемент (с высшим приоритетом)
3. Показать содержимое
4. Назад
Выберите действие: 1
Введите название объекта: A
Введите приоритет (целое число): 0
Элемент "A" (приоритет 0) добавлен.

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить элемент с приоритетом
2. Извлечь элемент (с высшим приоритетом)
3. Показать содержимое
4. Назад
Выберите действие: 1
Введите название объекта: B
Введите приоритет (целое число): 1
Элемент "B" (приоритет 1) добавлен.

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить элемент с приоритетом
2. Извлечь элемент (с высшим приоритетом)
3. Показать содержимое
4. Назад
Выберите действие: 3
```

```
--- Содержимое приоритетной очереди ---
Имя: B | Приоритет: 1
Имя: A | Приоритет: 0
-----

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить элемент с приоритетом
2. Извлечь элемент (с высшим приоритетом)
3. Показать содержимое
4. Назад
Выберите действие:
```

2.Реализовали структуру данных очередь:

```
--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить элемент (Enqueue)
2. Извлечь элемент (Dequeue)
3. Показать содержимое
4. Назад
Выберите действие: 1
Введите название объекта: А
Элемент "А" добавлен в очередь.

--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить элемент (Enqueue)
2. Извлечь элемент (Dequeue)
3. Показать содержимое
4. Назад
Выберите действие: 1
Введите название объекта: В
Элемент "В" добавлен в очередь.

--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить элемент (Enqueue)
2. Извлечь элемент (Dequeue)
3. Показать содержимое
4. Назад
Выберите действие: 3
Содержимое очереди (FIFO) ---
Имя: А
Имя: В
-----
```

3.Реализовали Стек:

```
--- СТЕК (LIFO) ---
1. Поместить элемент (Push)
2. Извлечь элемент (Pop)
3. Показать содержимое
4. Назад
Выберите действие: 1
Введите название объекта: А
Элемент "А" помещен в стек.

--- СТЕК (LIFO) ---
1. Поместить элемент (Push)
2. Извлечь элемент (Pop)
3. Показать содержимое
4. Назад
Выберите действие: 1
Введите название объекта: В
Элемент "В" помещен в стек.

--- СТЕК (LIFO) ---
1. Поместить элемент (Push)
2. Извлечь элемент (Pop)
3. Показать содержимое
4. Назад
Выберите действие: 3
Содержимое стека (LIFO) ---
Имя: В
Имя: А
-----
```

2. Листинг

Файл main.cpp

```
#include <iostream>

#include <string>

#include <locale>

using namespace std;

struct PriorityNode {

    string info;

    int priority;

    PriorityNode* next;

    PriorityNode(const string& val, int prio) : info(val),
priority(prio), next(nullptr) {}

};

struct Node {

    string info;

    Node* next;

    Node(const string& val) : info(val), next(nullptr) {}

};

class PriorityQueue {

private:

    PriorityNode* head;
```

```
public:
```

```
    PriorityQueue() : head(nullptr) {}
```

```
~PriorityQueue() {
```

```
    while (head != nullptr) {
```

```
        PriorityNode* temp = head;
```

```
        head = head->next;
```

```
        delete temp;
```

```
    }
```

```
}
```

```
void enqueue(const string& val, int priority) {
```

```
    PriorityNode* newNode = new PriorityNode(val, priority);
```

```
    if (head == nullptr || priority > head->priority) {
```

```
        newNode->next = head;
```

```
        head = newNode;
```

```
        cout << "Элемент \"" << val << "\" (приоритет " << priority  
<< ") добавлен.\n";
```

```
        return;
```

```
    }
```

```
    PriorityNode* current = head;
```

```
    while (current->next != nullptr && current->next->priority >=  
priority) {
```

```
        current = current->next;
```

```
    }
```

```

        newNode->next = current->next;

        current->next = newNode;

        cout << "Элемент \"" << val << "\" (приоритет " << priority <<
        ") добавлен.\n";

    }

```

```

void dequeue() {

```

```

    if (head == nullptr) {

        cout << "Приоритетная очередь пуста!\n";

        return;

    }

```

```

        PriorityNode* temp = head;

        cout << "Извлечен элемент: " << temp->info << " (приоритет: "
        << temp->priority << ")\n";

        head = head->next;

        delete temp;

    }

```

```

void display() const {

```

```

    if (head == nullptr) {

        cout << "Приоритетная очередь пуста.\n";

        return;

    }

```

```

    cout << "\n--- Содержимое приоритетной очереди ---\n";

```

```

    PriorityNode* current = head;

```

```

    while (current != nullptr) {

```

```

        cout << "Имя: " << current->info << " | Приоритет: " <<
current->priority << "\n";

        current = current->next;

    }

    cout << "-----\n";

}

};

```

```

class Queue {
private:
    Node* head;

    Node* tail;

public:
    Queue() : head(nullptr), tail(nullptr) {}

    ~Queue() {
        while (head != nullptr) {
            Node* temp = head;
            head = head->next;
            delete temp;
        }
    }

    void enqueue(const string& val) {
        Node* newNode = new Node(val);

        if (tail == nullptr) {
            head = tail = newNode;

```

```

    }

    else {

        tail->next = newNode;

        tail = newNode;

    }

    cout << "Элемент \"" << val << "\"" добавлен в очередь.\n";

}

```

```

void dequeue() {

    if (head == nullptr) {

        cout << "Очередь пуста!\n";

        return;

    }

    Node* temp = head;

    cout << "Извлечен элемент: " << temp->info << "\n";

    head = head->next;

    if (head == nullptr) {

        tail = nullptr;

    }

    delete temp;

}

```

```

void display() const {

    if (head == nullptr) {

        cout << "Очередь пуста.\n";

        return;

    }

}

```

```

        cout << "\n--- Содержимое очереди (FIFO) ---\n";

        Node* current = head;

        while (current != nullptr) {

            cout << "Имя: " << current->info << "\n";

            current = current->next;

        }

        cout << "-----\n";

    }

};

```

```

class Stack {

private:

    Node* top;

public:

    Stack() : top(nullptr) {}

    ~Stack() {

        while (top != nullptr) {

            Node* temp = top;

            top = top->next;

            delete temp;

        }

    }

    void push(const string& val) {

        Node* newNode = new Node(val);

```

```

newNode->next = top;

top = newNode;

cout << "Элемент \"" << val << "\"" помещен в стек.\n";

}

```

```

void pop() {
    if (top == nullptr) {
        cout << "Стек пуст!\n";
        return;
    }

    Node* temp = top;

    cout << "Извлечен элемент: " << temp->info << "\n";

    top = top->next;

    delete temp;
}

```

```

void display() const {
    if (top == nullptr) {
        cout << "Стек пуст.\n";
        return;
    }

    cout << "\n--- Содержимое стека (LIFO) ---\n";

    Node* current = top;

    while (current != nullptr) {
        cout << "Имя: " << current->info << "\n";

        current = current->next;
    }
}

```

```

    }

    cout << "-----\n";

}

};

int main() {

    setlocale(LC_ALL, "rus");

    PriorityQueue pQueue;

    Queue queue;

    Stack stack;

    int choice = 0;

    while (choice != 4) {

        cout << "\n===== МЕНЮ ЛАБОРАТОРНОЙ РАБОТЫ №3 =====\n";

        cout << "1. Работа с Приоритетной очередью\n";

        cout << "2. Работа с Обычной очередью (FIFO)\n";

        cout << "3. Работа со Стекком (LIFO)\n";

        cout << "4. Выход\n";

        cout << "Выберите структуру данных: ";

        cin >> choice;

        if (choice == 1) {

            int subChoice = 0;

            while (subChoice != 4) {

                cout << "\n--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---\n";

                cout << "1. Добавить элемент с приоритетом\n";

                cout << "2. Извлечь элемент (с высшим приоритетом)\n";

                cout << "3. Показать содержимое\n";

                cout << "4. Назад\n";

```

```

        cout << "Выберите действие: ";
        cin >> subChoice;

        if (subChoice == 1) {
            string name;
            int prio;
            cout << "Введите название объекта: ";
            cin >> name;
            cout << "Введите приоритет (целое число): ";
            cin >> prio;
            pQueue.enqueue(name, prio);
        }
        else if (subChoice == 2) {
            pQueue.dequeue();
        }
        else if (subChoice == 3) {
            pQueue.display();
        }
    }
}

else if (choice == 2) {
    int subChoice = 0;
    while (subChoice != 4) {
        cout << "\n--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---\n";
        cout << "1. Добавить элемент (Enqueue)\n";
        cout << "2. Извлечь элемент (Dequeue)\n";
        cout << "3. Показать содержимое\n";
        cout << "4. Назад\n";
        cout << "Выберите действие: ";
        cin >> subChoice;
    }
}

```

```

        if (subChoice == 1) {
            string name;
            cout << "Введите название объекта: ";
            cin >> name;
            queue.enqueue(name);
        }
        else if (subChoice == 2) {
            queue.dequeue();
        }
        else if (subChoice == 3) {
            queue.display();
        }
    }
}

else if (choice == 3) {
    int subChoice = 0;
    while (subChoice != 4) {
        cout << "\n--- СТЕК (LIFO) ---\n";
        cout << "1. Поместить элемент (Push)\n";
        cout << "2. Извлечь элемент (Pop)\n";
        cout << "3. Показать содержимое\n";
        cout << "4. Назад\n";
        cout << "Выберите действие: ";
        cin >> subChoice;

        if (subChoice == 1) {
            string name;
            cout << "Введите название объекта: ";
            cin >> name;

```

```
        stack.push(name);
    }
    else if (subChoice == 2) {
        stack.pop();
    }
    else if (subChoice == 3) {
        stack.display();
    }
}

}

}

cout << "Программа завершена.\n";
return 0;

}
```